Trabalho Prático - Percepção e Navegação Autônoma de um Robô Móvel

Marcelo Alves Batista * **Gabriel Marlon Viana** *

* *Faculdade de Engenharia Mecatrônica, Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG)*

1. INTRODUÇÃO E OBJETIVOS

Este trabalho tem como objetivo consolidar os conceitos teóricos e práticos de percepção, comunicação e navegação de robôs móveis através da implementação de uma missão autônoma no robô simulado `rm_bot`. O robô deve partir da origem $S = (0, 0)$ e visitar sequencialmente os destinos $P_1(1, 0)$, $P_2(3, 0)$ e $P_3(3, 2)$.

Durante o trajeto, o sistema precisa usar seus sensores embarcados para detectar e desviar de caixas estáticas colocadas no caminho, mantendo o destino salvo na memória e parando completamente ao finalizar o percurso. Este relatório detalha a arquitetura do sistema, o processamento dos dados do sensor LiDAR e as estratégias de controle aplicadas para a movimentação e prevenção de colisões.

2. METODOLOGIA

2.1 Arquitetura, Cenário e Parâmetros

O sistema foi desenvolvido utilizando os nós nativos do ROS 2. Analisando o fluxo de dados com a ferramenta `rqt_graph` (Figura 1), notamos que o controle do robô depende principalmente do tópico `/scan` (`sensor_msgs/LaserScan`) para ler o ambiente, e do tópico `/odom` (`nav_msgs/Odometry`) para estimar a posição do robô. Os comandos de velocidade gerados pelo nosso algoritmo são enviados para o tópico `/cmd_vel_auto`, que repassa a informação para o controle das rodas em `/cmd_vel` (`geometry_msgs/Twist`).

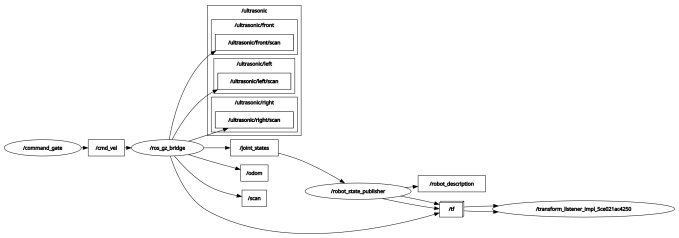


Figura 1. Grafo do ROS 2 (`rqt_graph`) indicando as conexões entre nós e tópicos na simulação.

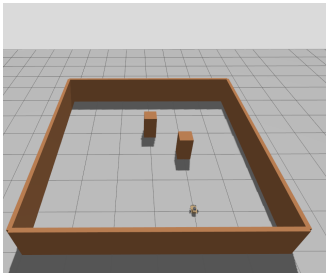
Os testes foram feitos no simulador Gazebo em dois cenários: o Cenário A (totalmente sem obstáculos) e o Cenário B, que possui duas caixas bloqueando o caminho direto entre os pontos de destino (ver Figuras 2 e 3). Usamos uma tolerância de chegada (ϵ) de 0,12 m, com velocidades máximas de $v_{\max} = 0,15$ m/s e $\omega_{\max} = 0,65$ rad/s. A Tabela 2 resume esses parâmetros e a posição dos elementos.

Tabela 1. Caracterização das interfaces de comunicação e tópicos do sistema.

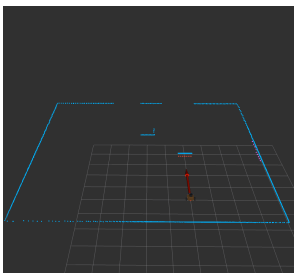
Tópico	**Tipo de Mensagem**
`/scan`	`sensor_msgs/LaserScan`
`/odom`	`nav_msgs/Odometry`
`/cmd_vel_auto`	`geometry_msgs/Twist`
`/cmd_vel`	`geometry_msgs/Twist`

Tabela 2. Configurações dos cenários e parâmetros.

Parâmetro / Elemento	**Valor / Especificação**
Pose Inicial S	(0,0 m, 0,0 m, 0,0 rad)
Metas P_1, P_2, P_3	$(1, 0)$, $(3, 0)$, $(3, 2)$ m
Caixa A (centro)	(2,0; 0,0; 0,40) m
Caixa B (centro)	(3,0; 1,0; 0,40) m
Dimensões Caixas	$0,40 \times 0,40 \times 0,80$ m
Tolerância (ϵ)	0,12 m
Limites ($v_{\max}, \omega_{\max}$)	0,15 m/s, 0,65 rad/s
Margem (r_{inf}) / Limite (d_{lim})	0,30 m/0,60 m



(a) Visualização no Gazebo.



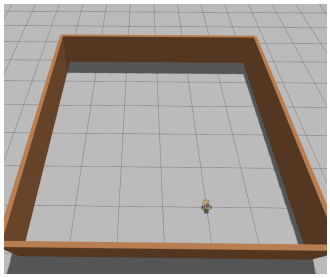
(b) Odometria no RViz.

Figura 2. Configuração do Cenário B (com obstáculos).

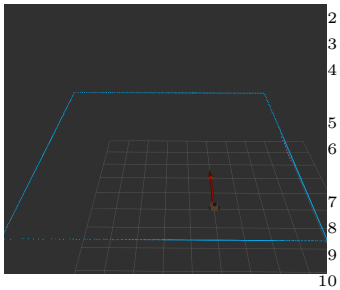
2.2 Processamento do Sensor LiDAR

Os dados do sensor foram divididos em três setores para facilitar a identificação dos obstáculos ao redor do robô: frontal ($-20°$ a $+20°$), esquerdo ($+20°$ a $+90°$) e direito ($-90°$ a $-20°$). Para evitar problemas caso o sensor fosse montado de forma diferente, os ângulos foram calculados dinamicamente com base nas propriedades da mensagem usando a Equação 1:

$$\theta_i = \theta_{min} + i\Delta\theta, \quad i = 0, \cdots, N-1 \tag{1}$$



(a) Visualização no Gazebo.



(b) Odometria no RViz.

Figura 3. Configuração do Cenário A (sem obstáculos).

Antes de tomar qualquer decisão de movimento, o sistema verifica se os dados do LiDAR são realmente confiáveis. A Listagem 1 mostra a validação da mensagem, descartando leituras vazias, dados desatualizados ou com erro nos limites de medição.

Listing 1: Validação de segurança e qualidade da mensagem LaserScan.

```
def on_scan(self, m):
    # Verifica se existem dados invalidos na estrutura da mensagem
    if (m.header.frame_id != 'lidar_link' or not m.ranges
        or not all(math.isfinite(v) for v in (m.angle_min, m.angle_increment, m.range_min, m.range_max))
        or m.angle_increment <= 0
        or m.range_max <= m.range_min
        or (len(m.ranges)-1)*m.angle_increment < 6.2):
            return

    # Salva o scan e atualiza o timer para controle de atraso
    self.scan = m
    self.scan_time = time.monotonic()
```

Um setor é classificado como `OCUPADO` se houver alguma leitura válida indicando algo mais próximo que o limite de proximidade (d_{lim}). O setor é considerado `LIVRE` apenas se não houver obstáculos próximos e o sensor tiver escaneado a área corretamente (com mais de 95% de dados válidos). Se os dados estiverem com muito ruído ou atrasados, o setor é marcado como `SEM_DADOS` por segurança.

2.3 Navegação Sequencial e Desvio Reativo

A navegação funciona acompanhando uma lista de destinos. O robô só avança para o próximo ponto quando a distância direta (euclidiana) até o alvo atual for menor ou igual à tolerância $\epsilon = 0,12$ m, conforme a Equação 2:

$$e_k = \sqrt{(x_k - x)^2 + (y_k - y)^2} \leq \epsilon \qquad (2)$$

A Listagem 2 ilustra o trecho responsável por verificar a chegada ao alvo. Ao chegar no ponto, as variáveis de controle de progresso são zeradas (evitando alertas falsos de que o robô está travado), e o próximo destino é carregado. Se for o último destino, a missão é finalizada.

Listing 2: Lógica de transição entre waypoints e encerramento da missão.

```
# Verificacao do criterio de chegada
if rho <= self.p['epsilon']:
    p_atual = self.waypoints[self.waypoint_index]
    self.get_logger().info(f'Chegou ao destino P{self.waypoint_index + 1}: {p_atual}')

    # Reset dos indicadores internos para o proximo trecho
    self.best_distance = math.inf
    self.progress_time = now
    self.previous_heading = 0.0

    # Avanca o indice para a proxima meta
    self.waypoint_index += 1

    if self.waypoint_index < len(self.waypoints):
        self.goal = self.waypoints[self.waypoint_index]
    else:
        # Encerramento da missao parando os motores
        self.goal = None
        self.command('MISSAO_CONCLUIDA', 0.0, 0.0)
    return
```

Para desviar das caixas, o sistema projeta um "corredor" imaginário à frente do chassi. A Listagem 3 demonstra a rotina geométrica `corridor_clear`, que testa ativamente se os pontos lidos pelo sensor invadem o espaço que o corpo do robô precisa para passar sem bater (margem de $r_{\text{inf}} = 0,30$ m).

Listing 3: Teste do corredor geométrico para garantir a passagem sem colisões laterais.

```
def corridor_clear(points, heading, length, radius):
    c, s = math.cos(heading), math.sin(heading)
    for x, y in points:
        # Projeta as leituras nas componentes de avanco e lateral
        along, across = x*c + y*s, -x*s + y*c
        nearest = max(0.0, min(length, along))

        # Verifica se o objeto entra no raio de seguranca da estrutura
        if math.hypot(along-nearest, across) < radius:
            return False
    return True
```

Durante a manobra de desvio, o destino original não é perdido. O robô avalia 18 direções possíveis entre o intervalo -90° a +90° para os lados e escolhe a direção mais livre que ainda aponte o máximo possível para o destino, evitando viradas bruscas. Assim que o obstáculo fica para trás e a rota direta é liberada, ele volta naturalmente a mirar no objetivo.

3. RESULTADOS EXPERIMENTAIS

Realizamos três ensaios no Gazebo sob as mesmas condições iniciais, variando apenas o cenário e o parâmetro de detecção. O Ensaio E1 foi executado no Cenário A (sem obstáculos). O Ensaio E2 testou o comportamento do sistema no Cenário B (com as duas caixas), usando o limite de proximidade padrão de $d_{\text{lim}} = 0,60$ m. O Ensaio E3 repetiu o Cenário B, mas aumentou o limite para

$d_{\text{lim}} = 0{,}80$ m para avaliar como isso afetaria a manobra de desvio.

As trajetórias reais realizadas pelo robô estão desenhadas nas Figuras 4, 5 e 6. Visualmente, percebe-se que no Ensaio E3 (com limite maior) o robô começou a curvar antes e fez uma manobra mais suave ao passar pela primeira caixa.

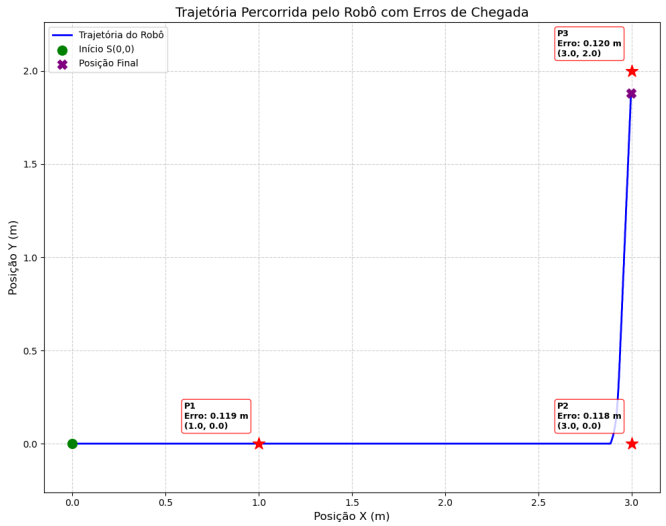


Figura 4. Trajetória $x - y$ no Ensaio E1 (Cenário A – Sem Obstáculos).

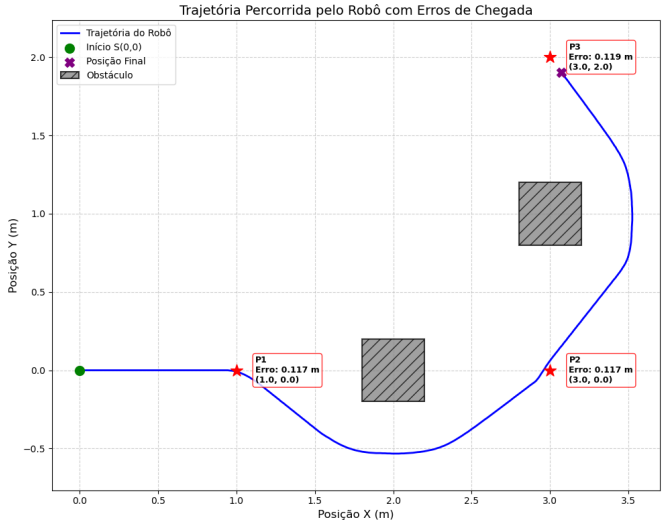


Figura 5. Trajetória $x - y$ no Ensaio E2 (Cenário B – $d_{\text{lim}} = 0{,}60$ m).

O histórico sequencial de ações, indicando quando o robô estava indo direto ao ponto ou quando estava desviando, está detalhado na Tabela 3. A Tabela 4 apresenta a precisão da chegada em cada meta e o tempo total dos percursos.

4. DISCUSSÃO

Os testes demonstram pontos importantes sobre a confiabilidade do nosso algoritmo. Primeiramente, confirmamos que usar apenas a informação do "menor obstáculo" do LiDAR não é uma boa estratégia de movimento. Saber

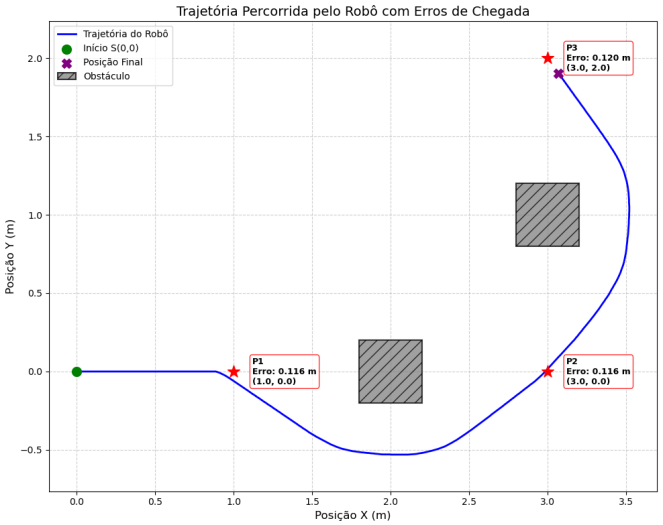


Figura 6. Trajetória $x - y$ no Ensaio E3 (Cenário B – $d_{\text{lim}} = 0{,}80$ m).

Tabela 3. Histórico de estados de navegação e tempos decorridos por ensaio.

Ensaio	Intervalo (s)	Estado (`log_state`)	Descrição do Evento
E1	$[0{,}0 - 8{,}2]$	`INDO_AO_PONTO`	Navegação direta rumo a P_1
	8,3	`CHEGOU_P1`	Alcançou P_1 ($\epsilon \leq 0{,}12$ m)
	$[8{,}4 - 22{,}6]$	`INDO_AO_PONTO`	Navegação direta rumo a P_2
	22,7	`CHEGOU_P2`	Alcançou P_2 ($\epsilon \leq 0{,}12$ m)
	$[22{,}8 - 38{,}0]$	`INDO_AO_PONTO`	Navegação direta rumo a P_3
	38,1	`CHEGOU_P3`	Missão concluída em P_3
E2	$[0{,}0 - 6{,}5]$	`INDO_AO_PONTO`	Navegação direta rumo a P_1
	6,6	`CHEGOU_P1`	Alcançou P_1 ($\epsilon \leq 0{,}12$ m)
	$[7{,}0 - 17{,}5]$	`DESVIANDO`	Contorno da Caixa A ($d_{\text{lim}} = 0{,}60$ m)
	$[17{,}6 - 23{,}1]$	`INDO_AO_PONTO`	Retomada da busca direta por P_2
	23,2	`CHEGOU_P2`	Alcançou P_2 ($\epsilon \leq 0{,}12$ m)
	$[23{,}3 - 34{,}4]$	`DESVIANDO`	Contorno da Caixa B ($d_{\text{lim}} = 0{,}60$ m)
	$[34{,}5 - 40{,}0]$	`INDO_AO_PONTO`	Retomada da busca direta por P_3
	40,1	`CHEGOU_P3`	Missão concluída em P_3
E3	$[0{,}0 - 6{,}5]$	`INDO_AO_PONTO`	Navegação direta rumo a P_1
	6,6	`CHEGOU_P1`	Alcançou P_1 ($\epsilon \leq 0{,}12$ m)
	$[6{,}7 - 17{,}8]$	`DESVIANDO`	Contorno antecipado da Caixa A (0,80 m)
	$[17{,}9 - 23{,}3]$	`INDO_AO_PONTO`	Retomada da busca direta por P_2
	23,4	`CHEGOU_P2`	Alcançou P_2 ($\epsilon \leq 0{,}12$ m)
	$[23{,}5 - 34{,}0]$	`DESVIANDO`	Contorno antecipado da Caixa B (0,80 m)
	39,8	`CHEGOU_P3`	Missão concluída em P_3

Tabela 4. Métricas de desempenho (Tempo Acumulado e Erro Euclidiano).

Ponto	Ensaio E1		Ensaio E2		Ensaio E3	
	Tempo (s)	Erro (m)	Tempo (s)	Erro (m)	Tempo (s)	Erro (m)
P_1	8,3	0,119	6,6	0,117	6,6	0,117
P_2	22,7	0,118	23,2	0,117	23,4	0,117
P_3	38,1	0,120	40,1	0,119	39,8	0,118
Total	**38,1 s**	–	**40,1 s**	–	**39,8 s**	–

que há algo próximo não informa a largura do objeto nem se o robô vai colidir de frente ou apenas tangenciar a caixa. A solução através da verificação de corredor (`corridor_clear`) corrigiu esse problema, pois cruza as distâncias com a largura física do robô.

O sistema também conseguiu diferenciar corretamente o que é espaço livre do que é falta de dados. Exigindo que mais de 95% dos dados da varredura fossem válidos, conseguimos evitar que um erro do sensor fosse lido como caminho seguro, parando o robô por precaução caso as falhas fossem grandes.

A alteração do parâmetro d_{lim} impactou diretamente o percurso. Ao aumentar o limite de proximidade de 0,60 m para 0,80 m (mudança de E2 para E3), o robô percebeu

a Caixa A mais cedo, antecipando a ação de desvio (no segundo 6,7 ao invés de 7,0). Isso abriu a curva, tornando-a mais segura e espaçosa sem causar perda significativa no tempo total da missão, que durou cerca de 40 segundos em ambos os casos.

Por fim, manter o alvo na memória ajudou muito. A penalidade matemática criada no código fazia o robô tentar se virar sempre para a rota de menor custo em direção ao objetivo. No entanto, vale destacar que o sistema é totalmente reativo. Se colocarmos o robô em um cenário mais complexo, como um corredor sem saída ou obstáculos em forma de "U", ele não conseguirá encontrar o caminho de volta sem um planejador de rota global atuando em paralelo.

5. CONCLUSÕES

O sistema desenvolvido cumpriu todos os requisitos propostos para o trabalho. A integração da leitura dividida por setores e a verificação matemática de um corredor de segurança garantiram que o robô contornasse caixas perfeitamente. Mesmo com as limitações conhecidas de uma navegação puramente reativa, a estratégia adotada mostrou-se altamente eficiente no cenário exigido, permitindo que o robô chegasse aos destinos com excelente precisão e sem nenhuma colisão.

REFERÊNCIAS

ALVES, D. **Notas de Aula e Slides da Disciplina de Robótica Móvel com ROS 2**. Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), Campus Divinópolis, 2026. Material didático não publicado.